

PROJETO DE TOPOLOGIA DE REDE (MODELO OSI)

Projeto Integrador (PI)

1. Objetivo

Projetar a topologia de rede, com base no modelo OSI, necessária para o funcionamento do aplicativo do Projeto Integrador, considerando a comunicação entre o usuário, a aplicação em C#, o servidor/API e o banco de dados.

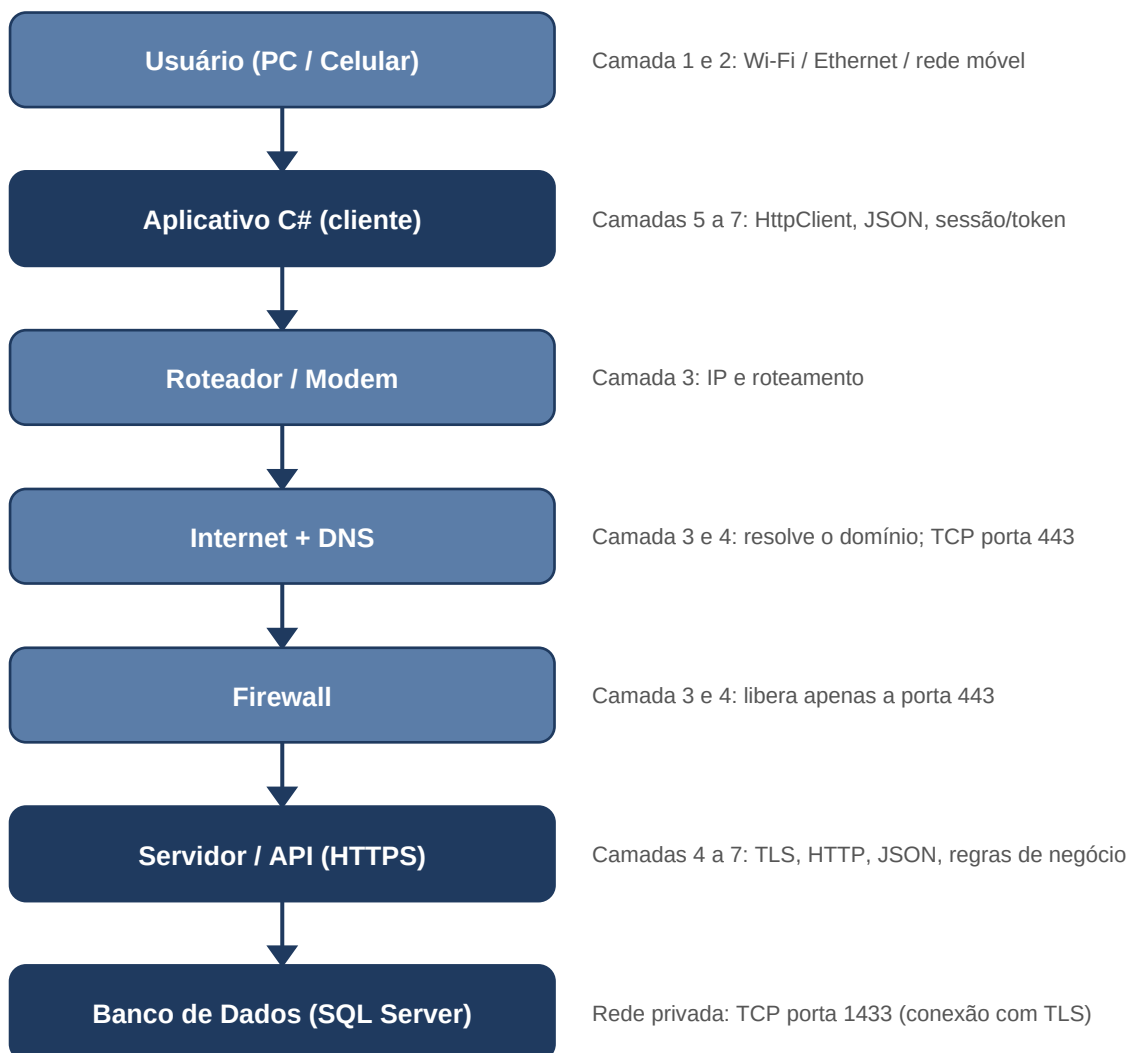
2. Topologia proposta

Componente	Função
Usuário (PC/Celular)	Acessa e utiliza o aplicativo.
Aplicativo em C#	Interface do usuário; monta e envia as requisições HTTP em formato JSON.
Roteador / Modem	Conecta o dispositivo à rede e encaminha os pacotes (IP) para a Internet.
Internet / DNS	Transporta os dados; o DNS converte o domínio da API em endereço IP.
Firewall	Filtra o tráfego e permite somente a porta 443 (HTTPS) para o servidor.
Servidor / API	Recebe as requisições, valida, processa os dados e acessa o banco.
Banco de Dados (SQL Server)	Armazena e consulta as informações. Fica em rede privada, acessível apenas pela API.

Observação: o SQL Server foi adotado como exemplo por ser o banco mais comum com C#. Se o projeto usar outro (MySQL, PostgreSQL), basta trocar o nome e a porta (3306 e 5432, respectivamente).

3. Diagrama da topologia

O fluxo segue do usuário até o banco de dados. A resposta faz o caminho inverso. À direita de cada elemento aparecem as camadas OSI envolvidas.



4. Mapeamento das camadas OSI

A tabela mostra como cada camada do modelo OSI aparece no projeto:

Camada	Nome	No projeto	Protocolos / tecnologias
7	Aplicação	O app C# envia requisições à API REST (cadastro, consulta etc.).	HTTP/HTTPS, API REST, HttpClient
6	Apresentação	Formato dos dados e criptografia do conteúdo.	JSON, TLS/SSL, UTF-8
5	Sessão	Mantém o usuário autenticado entre as requisições.	Token de autenticação (JWT), sessão TLS
4	Transporte	Entrega confiável entre o app e o servidor e entre a API e o banco.	TCP: porta 443 (API) e 1433 (SQL Server)
3	Rede	Endereçamento e roteamento dos pacotes pela Internet.	IP (IPv4/IPv6), DNS, roteador, firewall
2	Enlace	Comunicação entre dispositivos na mesma rede local.	Wi-Fi (802.11), Ethernet, endereço MAC, switch
1	Física	Transmissão dos sinais pelo meio físico.	Cabos, fibra óptica, ondas de rádio, placa de rede

5. Funcionamento da comunicação

O usuário abre o aplicativo C# no computador ou celular. Ao realizar uma ação, o aplicativo monta uma requisição HTTPS em JSON (camadas 7 e 6). A conexão TLS protege o conteúdo e o token mantém a sessão (camadas 6 e 5). O TCP leva os dados até a porta 443 (camada 4) e o IP os roteia pela Internet, após o DNS descobrir o endereço do servidor (camada 3). Os pacotes trafegam por Wi-Fi ou cabo (camadas 2 e 1). O firewall libera somente a porta 443. A API valida os dados e consulta o banco pela rede privada. A resposta retorna pelo mesmo caminho, em JSON.

6. Comunicação utilizando JSON

O aplicativo envia e recebe informações em JSON. Exemplo de corpo de requisição:

```
{
  "nome": "Caio",
  "email": "caio@email.com"
}
```

7. Implementação em C#

Exemplo de chamada POST com HttpClient (camada 7) e serialização JSON (camada 6):

```
using System.Net.Http.Json;

var client = new HttpClient();

var dados = new
{
    nome = "Caio",
    email = "caio@email.com"
};

var resposta = await client.PostAsJsonAsync(
    "https://exemplo.com/api/usuarios", dados);

var resultado = await resposta.Content.ReadAsStringAsync();
Console.WriteLine(resultado);
```

8. Segurança

- **HTTPS/TLS** em toda a comunicação entre o aplicativo e a API (camada 6).
- **Firewall** liberando apenas a porta 443 para o servidor (camadas 3 e 4).
- **Banco em rede privada**, sem acesso direto pela Internet; somente a API se conecta a ele.
- **Validação** dos dados recebidos e **controle de acesso** por token (camada 5).
- **Credenciais** em variáveis de ambiente ou arquivo de configuração, nunca no código-fonte.

9. Estrutura sugerida do projeto no GitHub

O projeto C# deve ficar na pasta SRC do repositório:

```
SEU-PROJETO/
|-- SRC/
|   |-- Program.cs
|   |-- Models/
|   |-- Services/
|-- README.md
```

10. Conclusão

A topologia proposta, analisada pelo modelo OSI, mostra o caminho completo dos dados: do aplicativo C# do usuário, passando pela rede local, Internet e firewall, até a API e o banco de dados. O uso de HTTPS e JSON garante segurança e facilita a integração entre os sistemas, e o banco em rede privada protege as informações armazenadas.